Communication Complexity: Communication Lower Bounds For Multi-Party Graph Traversal

Ryan Batubara rbatubara@ucsd.edu
Ivy Hawks m1hawks@ucsd.edu
Darren Ho rbatabura@ucsd.edu
Ciro Zhang ciz001@ucsd.edu
Dr. Shachar Lovett shachar.lovett@gmail.com

UC San Diego
HALICIOĞLU DATA SCIENCE INSTITUTE

cm cm

Communication Complexity: Communication Lower Bounds For Multi-Party Graph Traversal

Ryan Batubara rbatubara@ucsd.edu
Ivy Hawks m1hawks@ucsd.edu
Darren Ho rbatabura@ucsd.edu
Ciro Zhang ciz001@ucsd.edu
Dr. Shachar Lovett shachar.lovett@gmail.com

UC San Diego
HALICIOĞLU DATA SCIENCE INSTITUTE

Motivation

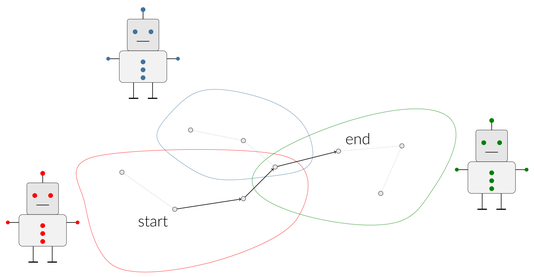

Figure 1. Path in Agent Owned Graphs

Communication Complexity

Communication Complexity studies how much information two (or more) parties must exchange in order to solve a problem when the input is split between them.

The key idea is that instead of asking: *How fast is the algorithm?*

We ask: *How many bits do the parties in the algorithm need to communicate?*

Two Party Communication Complexity

There are two players, traditionally called **Alice** and **Bob**.

- Alice holds input x
- Bob holds input y

Neither of them knows the inputs of the other persons, so they must talk back and forth until they know enough to compute some function $f(x, y)$

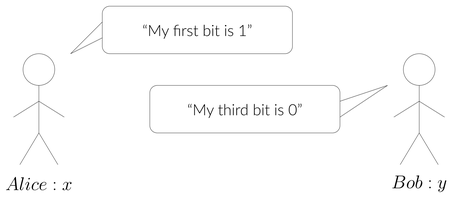

Figure 2. Alice and Bob exchanging messages.

In the worst case, someone just sends everything (n bits). So any problem that still requires n bits is **maximally hard**.

Equality is Maximally Hard

Equality Problem: Do Alice and Bob have the same n-bit string?

$$\mathrm{EQ}(x, y) = \begin{cases} 1 & x = y \\ 0 & x \neq y \end{cases}$$

In figure 3, we represent EQ as a communication matrix:

- Rows = Alice's input x
- Columns = Bob's input y
- Blue dots mark where $\mathrm{EQ}(x, y) = 1$

In communication complexity, a correct protocol must divide this matrix into rectangles that contain either all 0's or all 1's.

But each blue dot is isolated, so in order to cover all 2^n diagonal 1's, the protocol needs at least 2^n rectangles. It turns out this implies that Equality requires at least n bits of communication.

Thus, Equality is maximally hard.

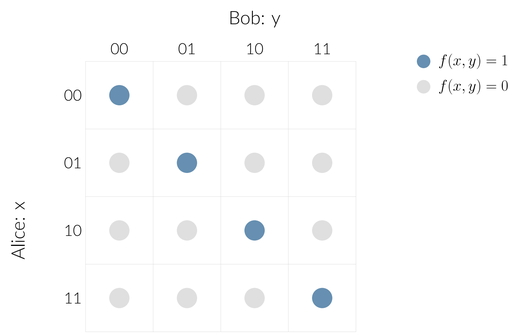

Figure 3. Communication matrix for Equality

The Traversal Problem

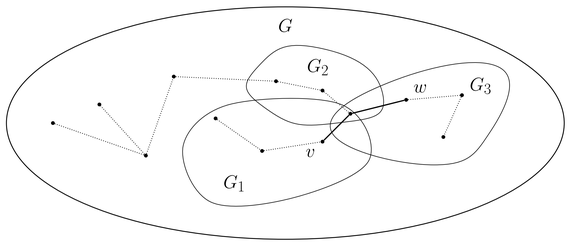

Figure 4. Traversal example where a valid path from v to w is shown in bold.

In this section, we define a simpler version of the problem from our motivation.

Consider a large graph G. Inside G, each agent "owns" part of the graph. We call these subgraphs $G_1, G_2, \ldots, G_k$. No single player sees the entire graph.

Given a starting vertex v and a target vertex w, how much information do the agents need to share about their subgraphs to determine whether there is a path from v to w?

For this poster, we will only consider a two agent case.

The Equality Gadget

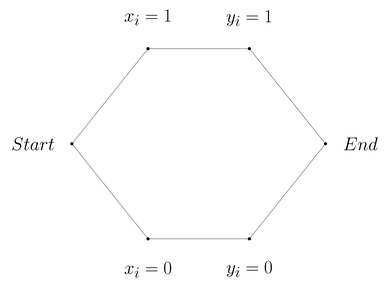

Figure 5. Gadget for proving equality Embedding

1. Top branch corresponds to $x_i = y_i = 1$,
2. Bot branch corresponds to $x_i = y_i = 0$.

Consider this gadget where its only Traversable from Start to End if both x_i and y_i are both 0 or 1. In other words when $y_i = x_i$

Embedding Equality into Traversal Equality Gadget Chain

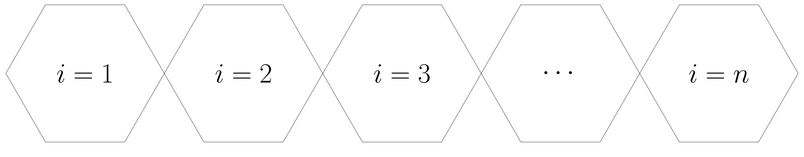

Figure 6. Chain of Equality Gadgets

To prove that Traversal, like Equality, is also maximally hard we show that we can embed Equality in Traversal.

By chaining the gadgets together, we turn each bit comparison into a small "gate." The chain is traversable only if every gate opens, which happens exactly when $x_i = y_i$ for all i, in other words, when $x = y$.

Therefore, any protocol that solves Traversal can also solve Equality. Since Equality is deterministically maximally hard, **Traversal is also deterministically maximally hard.**

Introducing Randomness

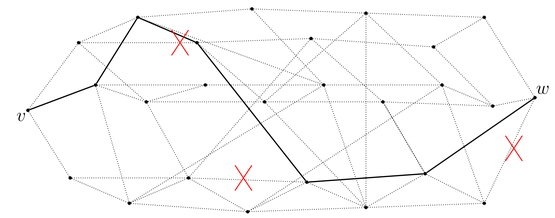

Figure 7. Graph with Skipped Edges

Deterministically we have seen that this is **maximally hard**. There is no better way to solve this problem than for Alice to send her entire input to Bob. However, we might also consider adding randomness, and allowing for error. For instance, if there is a path, it is unlikely that any randomly selected vertex will fall on it, so we might consider randomly removing some.

Set Disjointness

To find a lower bound for a random algorithm we can no longer rely on embedding equality, as random algorithm's for equality can be bounded as low as $\Theta(1)$. Thus, we turn to set disjointness.

$$\mathrm{DISJ}(x, y) = \begin{cases} 0 & x \cap y \neq \emptyset \\ 1 & x \cap y = \emptyset \end{cases}$$

Set Disjointness gives each player some number of elements from a set, and asks if the players have any elements in common. If so, the protocol outputs 0, and 1 otherwise. Set Disjointness is usefull when studying random algorithms since it has been shown that it requires **maximal *random* communication**.

Disjoint Gadget and Embedding

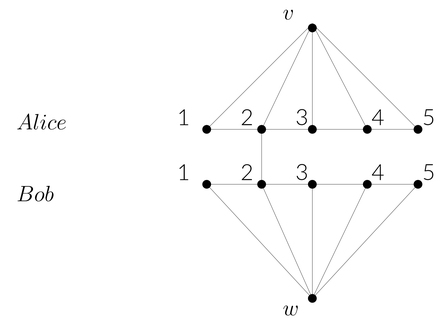

Figure 8. Disjointness Gadget. Here, a path exists as Alice and Bob both share element 2

This gadget can also be embedded into Traversal. Here, edges between Alice's side and Bob's side only exist if both players have an element in common